Healthcare RAG Report

Alex Tran

GitHub: github.com/alexthefirst99/Healthcare-RAG-Pipeline

Data

- **diabetes.pdf** — CDC “4 Steps to Manage Your Diabetes for Life” (a patient booklet), 20 pages.
- **standards.pdf** — “2025 ADA Standards of Medical Care in Diabetes: Updates!”, 11 pages. It has tables about blood sugar targets, emergency care, and foot/nerve checks.

Both were given as course materials for this assignment.

How It’s Built

Turning text into numbers (embeddings): Both systems use the same model, `all-MiniLM-L6-v2`, to turn text into numbers so the computer can compare meaning. Using the same model for both keeps things fair — any difference in results comes from how the text was cut up, not from a smarter or weaker “brain.”

Answering the question (generation): Both systems use the same AI model, `gpt-4o-mini`, to write the final answer. Note: the assignment’s example code never actually asks an AI to answer — it just prints back the raw text it found. I connected both systems to a real AI model so they’d give real answers, with the same rule for both: “only answer from the text you were given, and say so if the answer isn’t there.”

Where the text is stored (vector database): Both use FAISS, a fast tool for finding the most similar pieces of text to a question.

Cutting the documents into pieces (chunking): - Custom code: cuts the text every 500 characters, no matter where a sentence or word ends. This made 82 pieces. - LangChain: also aims for 500 characters, but tries to stop at the end of a sentence or paragraph first. This made 100 pieces (more, smaller pieces, since it often stops early).

A version problem I ran into: The assignment’s LangChain example code uses a shortcut called `RetrievalQA`. That shortcut doesn’t exist anymore — LangChain removed it in a recent update. So I had to write the “ask the AI a question with context” step by hand instead of using the shortcut. This turned out to matter for the reflection below.

Part 3: Test Results (Top 3 matches)

#	Question	Custom Time	Custom Score	LangChain Time	LangChain Score
1	What are metformin side effects?	0.97s	3/5	0.76s	3/5
2	A1C target range for type 2 diabetes?	1.27s	5/5	1.07s	3/5
3	How to treat hypoglycemia?	0.75s	3/5	0.78s	3/5
4	When to check blood glucose?	1.04s	5/5	1.05s	5/5
5	Foot care recommendations for diabetics?	0.91s	2/5	1.54s	5/5
Average		**0.99s**	**3.6/5**	**1.04s**	**3.8/5**

I graded these myself by reading the actual retrieved text and the final answer for each of the 10 runs (5 questions × 2 systems), using this rule: **3** for an honest "I don't know" when the answer genuinely wasn't in the retrieved text (safe, but not helpful), **5** for a correct, specific answer, and lower scores for answers that were wrong or misleadingly vague.

Part 4: What I Learned

1. Which one found better answers? LangChain edged ahead overall, 3.8 vs. 3.6 — but the interesting part is *why*, not the final number. The two systems don't have a "better" and "worse" version; they have different strengths that happened to show up on different questions. On the A1C question, Custom found the exact target range (7-7.5%) and LangChain gave a vague, disconnected answer instead ("below 7," no specific range) — Custom clearly wins there, 5 vs. 3. On the foot-care question, the opposite happened: LangChain found the actual nerve-damage screening and treatment steps, while Custom missed that paragraph completely and said "I don't know" — LangChain clearly wins there, 5 vs. 2. LangChain's overall average is only higher because that second gap (3 points) was bigger than the first gap (2 points) — not because it's the stronger system across the board. **Lesson: averaging hides which system is actually better at what. Two systems can trade wins on different questions and still end up close in the final number, which tells you almost nothing about where each one's real strengths and weaknesses are.**

2. Which code is simpler? My custom code is 58 lines. The LangChain code is 49 lines. That's closer than the assignment expects (it guesses 48 vs. 12), because I had to add the "ask the AI" step to both — the assignment's original code never actually did that step in either version. The real difference isn't line count, it's what happens when things change: LangChain's old one-line shortcut for "search and answer" got removed in a software update, so today's LangChain code needs almost as many manual steps as writing it yourself. **When to use which:** write it yourself for a small project you want full control over. Use LangChain when you want ready-made tools for lots of different data sources and file types, and you're OK keeping up with its updates.

3. Did the cutting-up process damage medical words? Yes, in my custom code. One chunk started mid-word: "…erwise healthy people can have…" — it had chopped off the front of the word "otherwise." LangChain avoided this because it tries to stop at the end of a sentence or word instead of cutting at a fixed number of characters. Neither system did well with the tables in the PDFs, though — tables of numbers (like blood sugar targets) got turned into a messy string of numbers and words with no structure, which would be hard for a doctor to read. That's not really a chunking problem — it happens earlier, when the PDF's table gets converted to plain text.

4. What did I learn by building it myself? Writing my own version forced me to actually understand steps that LangChain normally hides: how "comparing two pieces of text" is done with math (cosine similarity), that adding new documents means rebuilding the whole search index from scratch (there's no simple "add one more file" option), and that "search" and "write the answer" are genuinely two separate steps you must connect yourself — even though LangChain used to make it look like one simple step.

5. What would I change before using this in a real hospital? (a) Cut documents by their actual structure — headings, paragraphs, tables — instead of a fixed character count, since a fixed cut can turn an important medical number table into confusing text. (b) Never trust one overall average by itself — Custom and LangChain each won big on a different question (Q2 vs. Q5) and roughly cancelled out in the average, so looking only at the final score would hide which system is actually better for which kind of question. A real deployment should track performance per question type, not just one combined number. (c) Require the AI to say exactly where its answer came from, and to admit when it doesn't know — this already happened sometimes in testing, and that behavior needs to be checked regularly, not just assumed. (d) Use a faster, more scalable search method once there are more than a couple of documents, since the current method checks every single chunk one by one. (e) Lock in a specific version of LangChain, since this assignment's own example code broke after a routine software update.

Demo: One Key Insight

The most useful thing I learned wasn't which system "won" — LangChain finished slightly ahead overall (3.8 vs. 3.6), but that number badly undersells what actually happened. Custom and LangChain each nailed a different question that the other one botched: Custom found the precise A1C target number that LangChain gave a vague answer for, while LangChain found the exact foot-care protocol that Custom

missed entirely. The two systems aren't "better" and "worse" — they have different strengths, and this particular set of 5 questions happened to weight LangChain's strength slightly more. In a healthcare setting, that matters: a system that's strong on numeric targets but weak on clinical protocols (or vice versa) needs to be known and planned for, not hidden behind one comfortable-looking average.